

AI224: OPERATING SYSTEMS

Lab 8

Objective In this lab, you are asked to understand and execute the Java code which is provided to you. This Java code is an implementation for the readers/writers classical synchronization problem with readers having priority. The readers and writers are basically implemented by their respective threads. In the readers and writers problem, a shared data structure, usually a file or a database, is accessed by reader threads and writer threads. For efficiency, many readers should be allowed to read simultaneously. For correctness, only one writer is allowed to write into the data structure at a time, all other threads (readers & writers) must wait. There exist three possible solutions. Each solution defines a kind of scheduling policy to decide which thread should first have access to the data structure as described below:

✎ *Readers have priority.* Once a reader has started reading, all newly-arriving readers are allowed to start reading (multiple readers can read at a time). A waiting writer must wait until all readers have finished. This means that a starvation may occur in cases where there is a steady stream of newly-arriving readers. You have been given the Java code that implements this solution:

`MainMethod.java`: Main Java class that creates reader and writer threads.

`Reader.java`: Reader Java class that defines the reader thread.

`Writer.java`: Writer Java class that defines the writer thread.

`Synch.java`: This Java class declares the semaphores (mutex and wrt) and counter (readcount) used to synchronize readers and writers.

`RandomSleep.java`: This Java class defines a “doSleep” method for suspending a thread.

✎ *Writers have priority.* Anytime there is a writer waiting, no new readers are allowed to start. In case there is a steady stream of newly-arriving writers, the readers may starve.

✎ *A starvation-free solution.* To avoid starvation it is necessary for readers to give priority to writers, and vice versa. In a busy system, this results in one batch of readers, followed by one writer, followed by all the readers that arrived in the meantime, followed by another writer etc.

Your task. Modify and adapt the given Java code that implements the Readers-having-priority solution so that it becomes a Java code that implements the writer-having-priority. Below are some directions that express the requirements of the new solution.

Directions. Here are some requirements for “giving writers priority”:

- ☞ Writers must have exclusive access: if a writer is active, no other writer or reader may start.
- ☞ Readers are active in parallel: if reader(s) are active, and no writer is waiting, a new reader is allowed to start.
- ☞ Once a reader or writer becomes active, it is allowed to finish. There is no way to interrupt active readers when a writer arrives: the writer has to wait until all these readers finish. The last reader that finishes should wake up the waiting writer.
- ☞ If a new reader arrives when a writer is waiting or a writer is active, the reader must wait.
- ☞ When a writer finishes, it wakes up the next waiting writer (if there is one), or it wakes up all the waiting readers (if there are any).
- ☞ When a reader finishes, it checks whether it is the last reader. If so, it wakes up the next waiting writer (if there is one).

Hints

1. In the new solution there will be two queues. The first waiting queue is for the readers who arrived when a writer was writing or when a writer was waiting. The second waiting queue is for the writers who arrived when at least one reader was reading the data structure or when a writer was writing on the data structure. The queues are part of the semaphore you use (i.e., one semaphore creates one semaphore queue).
2. You need to have multiple counters to count how many readers and writers are active (reading/writing) and/or waiting (in the queue).
3. Counters are shared variables among readers and writer i.e., they are subject to race condition and hence should be protected when manipulated at a given time by a given thread.
4. For a reader to read, it has to check whether there are waiting writers or an active writer already there. If none is there, it starts reading. Then, when it finishes, if it is the last reader, it checks whether a writer is waiting to wake it up. However, for a writer to write, it has to check whether some readers are reading or a writer is writing. Then, when it finishes writing, it wakes up a writer if there is any, otherwise it wakes up all waiting readers if there are any.